Chapter 12: Scope and Closures

Suppose we want a function that adds one to whatever number it receives. We already know how to write that. Now suppose we want to choose the amount to add while the program is running, and keep the resulting function:

```
function makeAdder(a) {
    return function(b) { return a + b };
};
addOne = makeAdder(1);
print(addOne(2));
```

In the functions chapter, this failed with `Unknown identifier: a`. `makeAdder(1)` returned a function, but the function retained only its parameters and body. When `addOne(2)` ran, its environment led back to the caller, where there was no `a` to find.

This chapter makes that program print `3`. The function keeps the environment in which it was created, and each call uses that environment as its parent. Function syntax stays the same. We also add an explicit `set` statement for updating an existing binding through that environment chain.

Lexical Scope

A name's **scope** determines where that name is visible. Under **lexical scope**, also called **static scope**, a function resolves an outer name through its defining environment. Under dynamic scope, it searches through the caller's environment instead. (1)

```
greeting = "Hello";
function showGreeting() { print(greeting) };
function frenchCaller() {
    greeting = "Bonjour";
    showGreeting();
};
frenchCaller();
```

This now prints `Hello`. `showGreeting` was created at the top level, where `greeting` is `"Hello"`. The local `greeting` in `frenchCaller` does not become an enclosing binding of `showGreeting` just because `frenchCaller` happens to call it.

Question	Dynamic scope	Lexical scope
Where does a call frame's parent lead?	To the caller's environment	To the function's defining environment
Which greeting does this program print?	`Bonjour`	`Hello`
Can a caller supply an otherwise missing outer name?	Yes	No

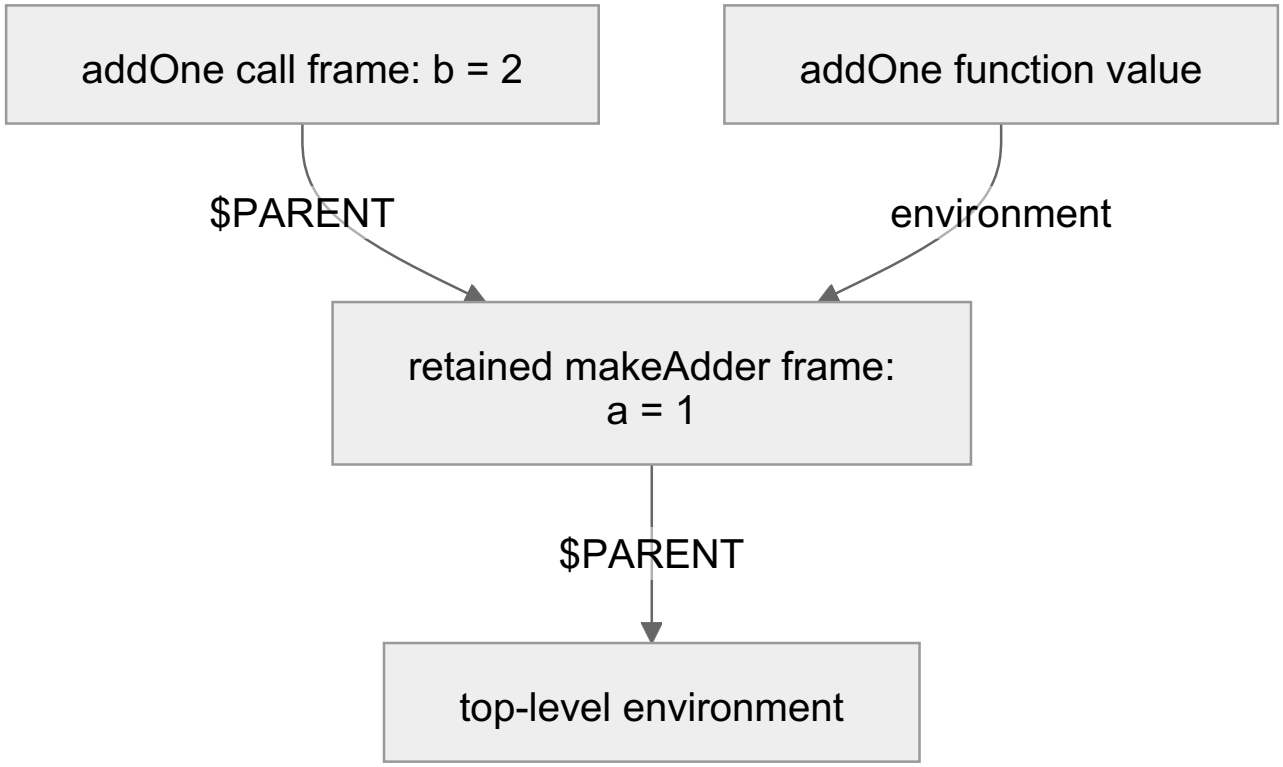
If the top-level assignment to `greeting` is removed, the lexical version raises `Unknown identifier: greeting`. Having a variable with the right name somewhere among the active callers is no longer enough.

A Function and Its Environment

In `function(b) { return a + b }`, `b` is a parameter. The function gets its binding from the argument supplied on each call. `a` is a **free variable** of that function: the body uses it without binding it locally. A **closure** combines a function with an environment that supplies its outer bindings. (2)

Evaluate `makeAdder(1)` one step at a time. The call creates an environment with `a` bound to `1`. Evaluating the inner function expression creates a function value that retains that environment. Returning the value leaves `addOne` referring to the function, and the function still referring to the environment containing `a`.

Calling `addOne(2)` creates another environment, this time with `b` bound to `2`. Its parent is the retained environment:



Looking up `b` stops in the current call frame. Looking up `a` follows one parent link. The addition produces `3`.

The functions chapter described a call frame as disappearing when its call returned. That was enough for functions that retained no environments. Now a call can finish while its environment remains reachable through a returned function. Finishing execution and becoming unreachable are two different events.

Separate Calls, Separate Closures

Calling the same factory twice creates two environments:

```
function makeAdder(a) {
    return function(b) { return a + b };
};
addOne = makeAdder(1);
addTen = makeAdder(10);
print(addOne(5));
print(addTen(5));
print(addOne(20));
```

This prints `6`, then `15`, then `21`. The two returned functions use the same body, but their `environment` fields lead to different calls of `makeAdder`. One has `a = 1`; the other has `a = 10`.

Each call of `addOne` also gets its own fresh frame for `b`. Retaining an outer environment does not mean reusing a function's own call frame. There are two jobs here: remember the outer bindings, and allocate the current call's parameters and local variables.

Shadowing and Local Assignment

A nearer binding **shadows** a farther one with the same name:

```
x = 100;
function makeReader(x) {
    return function() { return x };
};
readSeven = makeReader(7);
print(readSeven());
print(x);
```

This prints `7`, then `100`. The retained `makeReader` frame supplies the nearer `x`. The top-level binding still exists; lookup simply stops before reaching it.

Shadowing does not erase or overwrite the outer binding. The parameter `x` and the top-level `x` are two bindings with the same spelling in different environments. While `readSeven` runs, lookup checks its fresh call frame, then the retained factory frame, and stops at `x = 7`. The top-level `x = 100` remains unchanged.

Environment searched by `readSeven()`	Binding of `x`	Action
Current reader-call frame	Absent	Follow `$PARENT`
Retained `makeReader(7)` frame	`7`	Return `7` and stop
Top-level environment	`100`	Not reached by this lookup

A local assignment can introduce shadowing during a call. Before that assignment, a read may find an outer binding. After the assignment, reads stop at the new local binding. The outer value has not changed.

Reading and assigning names remain different operations. An ordinary assignment writes into the current environment:

```
x = 10;
function changeLocally() {
    x = x + 1;
    return x;
};
print(changeLocally());
print(changeLocally());
print(x);
```

This prints `11`, then `11`, then `10`. On each call, the right-hand `x` is read before a local `x` has been assigned, so lookup reaches the top level. Assignment then stores `11` in the current call's frame. The next call starts with another fresh frame.

Lexical scope changes where reads search. It does not turn every assignment into an update of the nearest existing binding. This version also resolves bindings as execution proceeds; it does not scan the whole function body in advance to classify local names. Python, for example, has its own rules for local binding and explicit `nonlocal` assignment; those rules should not be assumed to apply to Vertex. (3)

Capturing an Environment Is Not Copying It

The closure retains the environment dictionary itself. It does not take a snapshot of the values present when the function was created:

```
amount = 1;
function currentAmount() { return amount };
amount = 9;
print(currentAmount());
```

This prints `9`. The function still reaches the same top-level environment, and the binding in that environment has changed.

Static scope therefore does not mean that every value is fixed before execution. The enclosing relationship determines where lookup goes; the binding found there can change while the program runs.

Copying the dictionary would change this behavior. It would also make recursive definitions harder: a named function is assigned into its environment only after its function expression has been evaluated. Retaining the original dictionary lets the function see that assignment.

Updating an Existing Binding with set

Ordinary assignment gives us local variables. Sometimes we want to change a variable that already belongs to an enclosing environment. We make that choice explicit:

```
x = 10;
function changeOuter() {
    set x = x + 1;
    return x;
};
print(changeOuter());
print(changeOuter());
print(x);
```

This prints `11`, then `12`, then `12`. Each call reads and updates the same top-level binding. Unlike ordinary assignment, `set` does not create a local `x`.

Operation	Where it writes	Can it create a variable?
`x = expression`	Current environment	Yes
`set x = expression`	Nearest environment already containing `x`	No
`state.value = expression`	Field of the object reached through `state`	Creates or updates an object field

The search starts in the current environment. If the name is absent, it follows `$PARENT` links until it finds a binding or runs out of environments. A current local or parameter therefore takes precedence over an outer binding:

```
x = 100;
function outer() {
    x = 10;
    function inner() { set x = x + 1 };
    inner();
    return x;
};
print(outer());
print(x);
```

This prints `11`, then `100`. The local `x` belonging to `outer` is the nearest binding. `set` stops there rather than continuing to the top level. `set` respects shadowing: if `inner` instead takes a parameter named `x`, that current parameter is the nearest binding and `set` updates it. It does not skip a local variable to reach an outer one. The search uses lexical parents, so a name belonging only to the caller does not become available.

If no environment contains the name, `set` raises `NameError` with a message such as `Cannot set undefined variable: missing`. The target must already exist before the right-hand expression is evaluated. Consequently, `set missing = input()` raises the error without reading input.

A Counter with a Captured Variable

A closure can now update a plain captured binding:

```
function makeCounter(start) {
    count = start;
    return function() {
        set count = count + 1;
        return count;
    };
};
first = makeCounter(0);
second = makeCounter(10);
print(first());
print(first());
print(second());
```

This prints `1`, then `2`, then `11`. Each factory call creates a separate `count` binding. Every call of `first` has a fresh local environment, but its `set` statement reaches the same retained factory environment. The binding survives because the returned function still refers to that environment.

Private State Through Shared Data

An ordinary assignment cannot rebind a captured outer name, but an object reached through that name can still be changed:

```
function makeCounter(start) {
    state = {"value": start};
    return function() {
        state.value = state.value + 1;
        return state.value;
    };
};
first = makeCounter(0);
second = makeCounter(10);
print(first());
print(first());
print(second());
```

This prints `1`, then `2`, then `11`. Each factory call creates a new state object. Each returned function reaches its own object through its retained environment. The field assignment updates that object rather than rebinding the name `state`.

Two functions created during one factory call can share the same state:

```
function makeAccount(start) {
    state = {"balance": start};
    return {
        "deposit": function(amount) {
            state.balance = state.balance + amount;
            return state.balance;
        },
        "balance": function() { return state.balance }
    };
};
account = makeAccount(20);
print(account.deposit(5));
print(account.balance());
```

Both prints produce `25`. The two function values retain the same factory environment, so both reach the same state object. Vertex code cannot inspect a function's internal `environment` field. The returned record exposes operations on the state without exposing a field that directly contains the state object.

These two approaches preserve state in different places. The earlier counter uses `set` to update a binding in its retained environment. This account uses field assignment to change a shared object reached through a binding. Neither requires ordinary assignment to change its local rule.

Building Objects from Closures

The account factory has already assembled the parts of an object: private state and a set of operations that work on it. The returned record names the operations; their closures retain access to the state. A factory call constructs a new instance. This use of local state and returned functions is also developed in *Structure and Interpretation of Computer Programs*. (4)

Here is the correspondence:

Object concept	Closure-based construction
Construction	Call the factory
Instance state	Data retained through that call's environment
Method	A returned function that accesses the retained state
Public interface	The functions the factory returns
Separate instances	Separate factory calls and their environments

No class declaration is needed. We are constructing this behavior from functions, lexical scope, and records, all of which the interpreter already understands.

Methods Sharing a Captured Binding

We can also build the public interface around a plain captured variable:

```
function makeCounter() {
    count = 0;
    return {
        "increment": function() {
            set count = count + 1;
            return count;
        },
        "value": function() { return count }
    };
};
a = makeCounter();
b = makeCounter();
print(a.increment());
print(a.increment());
print(a.value());
print(b.value());
```

This prints `1`, then `2`, then `2`, then `0`. Both methods in `a` retain the same factory environment. One updates its `count` binding, and the other reads it. The methods in `b` retain a different environment with a separate binding. The public record exposes only the two functions.

Returning Capabilities

A returned function can be treated as a **capability**: possessing it lets the recipient perform a particular operation on the captured state. The factory need not give every recipient the entire interface:

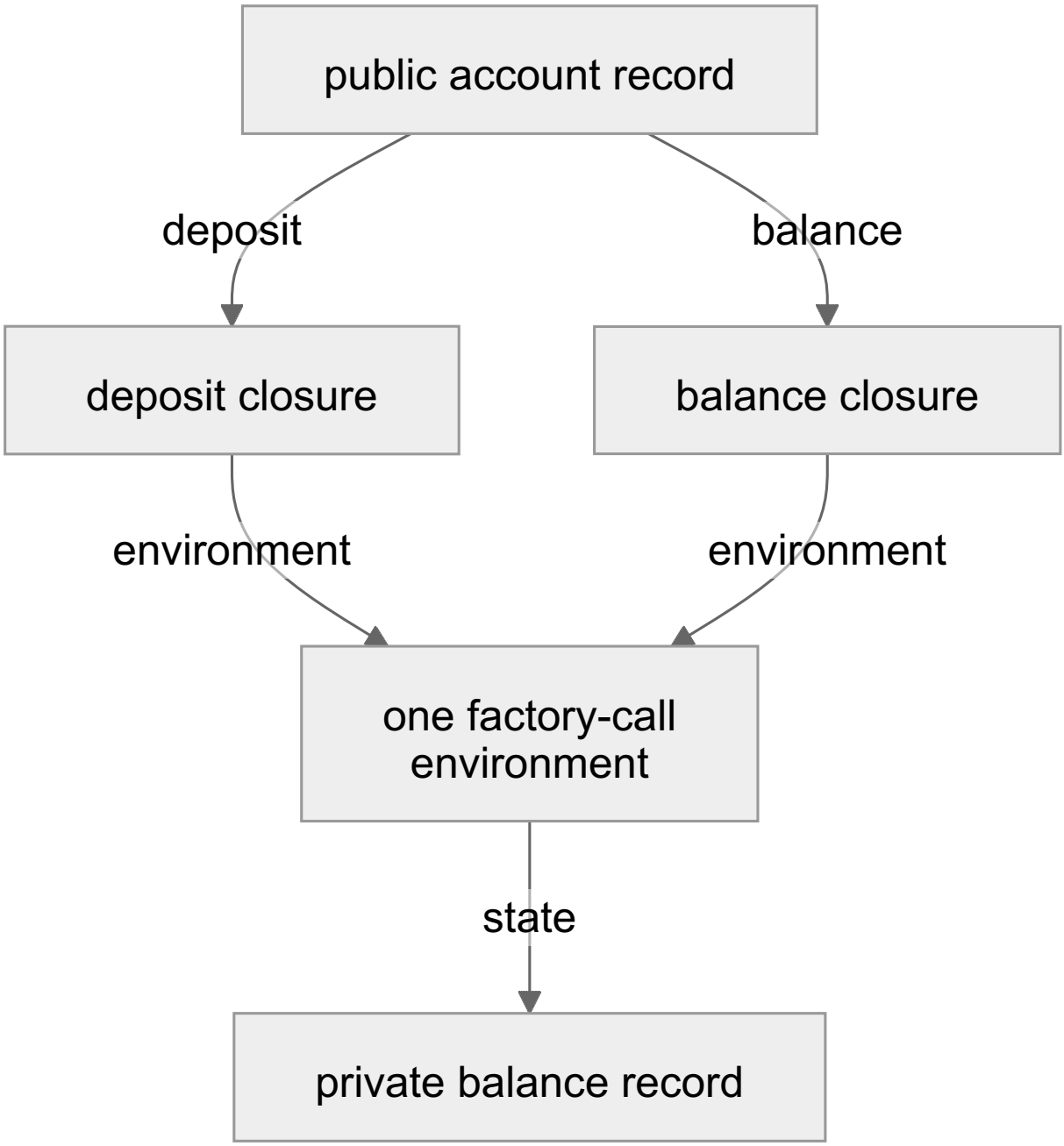
```
function makeAccount(start) {
    state = {"balance": start};
    return {
        "deposit": function(amount) {
            state.balance = state.balance + amount;
            return state.balance;
        },
        "balance": function() { return state.balance }
    };
};
first = makeAccount(20);
second = makeAccount(100);
readBalance = first.balance;
deposit = first.deposit;
print(deposit(5));
print(readBalance());
print(second.balance());
```

This prints `25`, then `25`, then `100`. `deposit` and `readBalance` retain access to the first account even after being extracted from its record. Calling either does not require passing that record back as a receiver. The second account has its own state.

Code given only `readBalance` can observe that account's balance through that function. It has not thereby received the deposit operation or a reference to the state object. The choices made by the factory determine what each returned function allows. In this example, deposit accepts any numeric amount, including a negative one; the name alone does not enforce a policy. Validation would belong inside the operation.

Where the Object Lives

There is no single new runtime value containing everything. The public record holds function values. Those values retain an environment, and a binding in that environment reaches the private state object:



The environment is still an environment, and the state record is still ordinary data. We have not made their lookup mechanisms interchangeable. The functions use identifier lookup to find `state`, then direct field access to read or change `state.balance`.

The public record itself remains mutable. Replacing `first.deposit` changes that record's interface; it does not rewrite an already extracted `deposit` closure or its captured state. Returning `state` itself would expose the data and allow callers to bypass the operations. Encapsulation here depends on which references the factory returns.

What This Construction Does Not Supply Automatically

These functions act as methods because they operate on a particular instance's state. Vertex has not added an implicit `this` parameter, class membership, inheritance, or special method-call rules. `first.deposit(5)` remains an ordinary field read followed by an ordinary function call. The closure remembers which state to use.

Functions created by separate factory calls are distinct function values, even when their bodies are identical. This straightforward implementation allocates new function values per instance. An object system could instead share method code and pass an explicit receiver; that would be a different implementation of the connection between behavior and state.

Changing the Evaluator

The function-expression case adds one field to the existing function value:

```
return {FUNCTION_KEY: function_count,
        "parameters": ast["parameters"],
        "body": ast["body"],
        "environment": environment}, None
```

This code runs when the function expression is evaluated, whether or not the resulting function is ever called. A function created at the top level retains the top-level environment. A function created inside a running function retains that call's environment.

The call case changes the source of the parent link:

```
call_environment = {PARENT_KEY: callee["environment"]}
for name, value in zip(callee["parameters"], arguments):
    call_environment[name] = value
value, status = evaluate(callee["body"], call_environment)
```

Previously, the parent was `environment`, the environment active at the call site. Now it is `callee["environment"]`, the one saved when the callee was created. Parameter binding and body evaluation proceed as before.

Identifier lookup needs no new search algorithm. It still checks the current dictionary and follows `$PARENT` links until it finds the name or runs out of environments. Changing the links changes the scope rule.

Recursion Still Works

```
function factorial(n) {
    if (n == 0) { return 1 };
    return n * factorial(n - 1);
};
print(factorial(6));
```

This still prints `720`. The function captures the top-level dictionary; the definition then assigns the function to `factorial` in that same dictionary. Every recursive call gets a fresh `n`, and every call's parent leads to the environment containing `factorial`.

The lexical parent chain does not list the active recursive calls. Python's evaluator calls still keep track of the computation waiting for each result. That execution history and the environment chain now serve visibly different purposes: one records where execution returns; the other determines where names are found.

Syntax, Evaluation Order, and Status

Function and call syntax stays unchanged. Their extra environment belongs to the runtime function value, not to the abstract syntax tree (AST).

The new statement has one grammar production:

```
set_statement ::= "set" identifier "=" expression
```

The tokenizer recognizes `set` as a keyword. Names such as `setter` remain identifiers. The parser requires a bare variable name and an equals sign, then uses the existing expression parser for the right-hand side. It returns a node containing the tag `"set"`, the name, and the expression. `set` is a statement, so it cannot appear inside an expression. Field and index assignments keep their existing syntax.

The evaluator first resolves the existing binding:

```
name = ast["name"]
destination = environment
while name not in destination:
    if PARENT_KEY not in destination:
        raise NameError(f"Cannot set undefined variable: {name}")
    destination = destination[PARENT_KEY]
value, status = evaluate(ast["expression"], environment)
if status is not None:
    return value, status
destination[name] = value
return value, None
```

The right-hand expression evaluates in the original current environment, even when the destination belongs to an outer one. A non-normal status propagates without updating the binding. Resolving the destination first also prevents a missing target from triggering right-hand-side effects.

The callee and arguments still evaluate in the caller's environment, with arguments evaluated left to right. Only the body runs in the new frame linked to the saved defining environment:

```
x = 3;
function addX(n) { return x + n };
function caller() {
    x = 20;
    return addX(x);
};
print(caller());
```

This prints `23`. The argument `x` supplies `20` from the caller. The `x` in `addX`'s body supplies `3` from its defining environment.

Arity checks and status handling are unchanged. A call consumes `return`, falling off the body produces `null`, and `exit` propagates outward. A stray `break` or `continue` cannot escape a function and control a loop belonging to its caller.

Verification

Run the chapter's example blocks through `python3 code/runner.py` from this chapter's directory. Each block above is self-contained. The first example must now print `3`; the caller-shadowing example must print `Hello`; separate counters must keep separate state.

Run `python3 -m unittest discover -s code` from this chapter's directory. The chapter-specific example checks execute every Vector block in this document and compare its output with the expected results. The `set` tests also cover nearest-binding lookup, undefined targets, local and parameter bindings, lexical rather than caller lookup, and shared state across methods and independent instances.

Considerations

1. A closure retains an environment rather than copying it. What does that choice make possible, and what changes can the closure observe?
2. Why does calling `makeAdder` twice create independent captured parameters, while creating two functions inside one call can give them shared state?
3. A retained environment may contain values the function never reads. What memory cost might that impose? What would an implementation have to determine before retaining only the needed bindings?
4. Why must arguments use the caller's environment even though the function body uses the defining environment?
5. Why does `set` update the nearest existing binding rather than always updating the top-level environment? Why resolve that binding before evaluating the right-hand expression?
6. If a caller receives only an account's balance-reading function, what can it do through that function? How would returning the state record as well change that answer?

Practice

1. Write `makeMultiplier(factor)`, returning a function of one argument. Use it to create `double` and `triple`.
2. Predict the output of the following program and explain each read of `x`:

```
x = 2;
function readX() { return x };
function caller() { x = 8; return readX() };
print(caller());
x = 5;
print(caller());
```

3. Write `makeStepper(start, step)`, returning a function that updates a private numeric value by `step` on each call and returns the new value. Use `set` to update a captured binding.
4. Change `makeCounter` so the returned value is a record containing `next` and `reset` functions. `reset` should restore the initial value.
5. Compare `count = count + 1`, `set count = count + 1`, and `state.value = state.value + 1` inside a returned function. Which binding or object does each operation change?

Answers to the Practice Questions

1. Something like:

```
function makeMultiplier(factor) {
    return function(n) { return factor * n };
};
double = makeMultiplier(2);
triple = makeMultiplier(3);
print(double(7));
print(triple(7));
```

   This prints `14`, then `21`.

2. It prints `2`, then `5`. `caller`'s local `x` does not enclose `readX`. Both reads in `readX` reach the same top-level environment, whose `x` changes between the calls.

3. Something like:

```
function makeStepper(start, step) {
    value = start;
    return function() {
        set value = value + step;
        return value;
    };
};
next = makeStepper(10, 3);
print(next());
print(next());
```

   This prints `13`, then `16`.

4. Something like:

```
function makeCounter(start) {
    count = start;
    return {
        "next": function() {
            set count = count + 1;
            return count;
        },
        "reset": function() { set count = start }
    };
};
counter = makeCounter(0);
print(counter.next());
print(counter.next());
counter.reset();
print(counter.next());
```

   This prints `1`, then `2`, then `1`.

5. Ordinary `count = count + 1` reads through lexical lookup but writes into the current call's environment. `set count = count + 1` updates the nearest existing `count`, including a captured binding. Field assignment changes the shared object reached through `state`; it does not rebind the name `state`.

References

1. Wikipedia contributors. "Scope (computer programming)." Wikipedia. https://en.wikipedia.org/wiki/Scope_(computer_science)
2. Wikipedia contributors. "Closure (computer programming)." Wikipedia. https://en.wikipedia.org/wiki/Closure_(computer_programming)
3. Python Software Foundation. "Execution model," sections "Binding of names" and "Resolution of names." Python Language Reference. https://docs.python.org/3/reference/executionmodel.html
4. Abelson, Harold, Gerald Jay Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*, section 3.1.1, "Local State Variables." Comparison Edition, with JavaScript adaptation. https://sicp.sourceacademy.org/chapters/3.1.1.html